Relatório do trabalho de algoritmos 2

Lilian Miranda Mendes – GRR 20222549

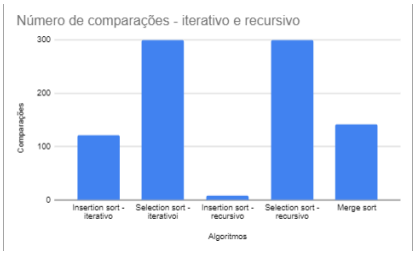
1. Introdução

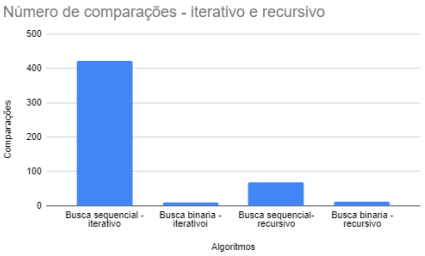
Relatório criado no Sistema Operacional Linux com a distribuição Ubuntu, utilizando a linguagem C para a criação dos algoritmos. O intuito foi de testar desempenho de alguns algoritmos recursivos de ordenação como Merge Sort, Insertion Sort, Selection Sort, enquanto os algoritmos de busca são Busca Binária, Busca Sequencial, todos com versão iterativa e recursiva (apenas o Merge Stort não foi implementado na versão iterativa), testando seu tempo de execução em segundos físicos e o número de comparações entre elementos de vetor. Foram feitos no total 6 testes, variando em conjuntos de dados, em tamanho e números de elementos. Os resultados incluem o tempo total de execução e o número total de comparações realizadas.

2. Resultado dos testes

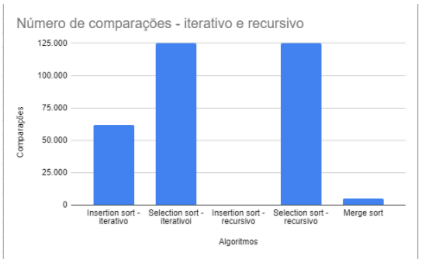
Ordenação e busca vetor de 25 posições:

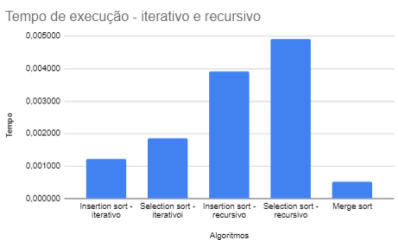
Ordenação e busca vetor de 100 posições:

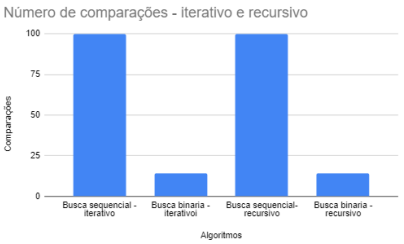




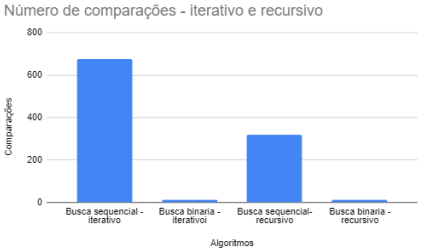
Ordenação e busca vetor de 500 posições:



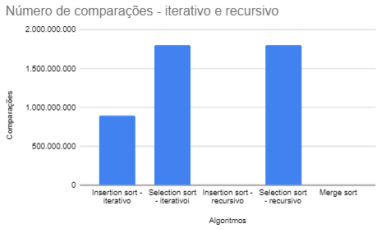




Ordenação e busca vetor de 1.000 posições:



Ordenação e busca vetor de 50.000 posições:



Ordenação e busca 100.000 posições:

Foram feitas tentativas com vetores de 500.000 posições, porém não foi possível coletar os dados por falha de segmentação.

3. Análise dos resultados

Com base nos gráficos é possível relatar o seguinte de cada algoritmo:

- Insertion sort:

O Insertion Sort iterativo geralmente supera a versão recursiva em eficiência de tempo e número de comparações, especialmente em tamanhos maiores de vetor. Isso se deve à natureza do algoritmo iterativo, que evita sobrecarga de chamadas recursivas e tem uma complexidade de tempo melhor controlada. Enquanto o tempo total aumenta exponencialmente na versão recursiva conforme o tamanho do vetor aumenta, o tempo do algoritmo iterativo cresce de maneira mais linear. O número de comparações na versão recursiva também tende a crescer mais rapidamente que na versão iterativa.

- Selection sort:

Assim como o Insertion Sort, o Selection Sort iterativo é mais eficiente que a versão recursiva, especialmente em tamanhos grandes de vetor. A abordagem iterativa do Selection Sort evita a sobrecarga associada às chamadas recursivas, resultando em um desempenho mais rápido e consistente. O tempo total e o número de comparações do Selection Sort iterativo são menores em comparação com a versão recursiva, principalmente em vetores maiores.

- Merge sort:

Sua eficiência é perceptível em todos os tamanhos de vetor, com tempos de execução baixos e um número relativamente pequeno de comparações. O tempo total do Merge Sort é muito menor em comparação com os algoritmos de ordenação iterativos para todos os tamanhos de vetor, e o número de comparações é significativamente menor em comparação com o Insertion e Selection Sort.

- Busca sequencial:

Não há diferença significativa entre a versão iterativa e recursiva da busca sequencial em termos de desempenho. Ambos realizam uma busca sequencialmente, com uma complexidade de tempo linear. O número de comparações na busca sequencial é diretamente proporcional ao tamanho do vetor, o que significa que cresce linearmente.

- Busca binária:

Tanto a versão iterativa quanto a recursiva da busca binária têm desempenho semelhante, com uma complexidade de tempo logarítmica. Ambas são significativamente mais eficientes que a busca sequencial, especialmente em vetores grandes. O número de comparações na busca binária é muito menor em comparação com a busca sequencial, tornando-a mais eficiente para vetores ordenados.

4. Conclusão

Em resumo, a escolha do algoritmo adequado para ordenação e busca depende de diversos fatores, como o tamanho do vetor, a estabilidade desejada e a ordenação prévia dos dados. Enquanto o Merge Sort destaca-se pela eficiência em todos os cenários, especialmente em vetores grandes, os algoritmos iterativos como Insertion e Selection Sorts oferecem uma abordagem mais eficaz para tarefas menores. Quanto às operações de busca, tanto a busca sequencial quanto a binária têm suas vantagens dependendo da estrutura do vetor e da necessidade de eficiência.